

Streaming Replication Recovery

Parameter-Mismatch Break & Standby Re-Sync

Scope	Diagnostics · Path A / B / C / D · Verification · Troubleshooting
Engine	PostgreSQL 16 — physical streaming replication
Topology	Primary (db01) → Standby (db02), 2 VMs
Slot / User	Slot <code>ssncpri</code> · replication user <code>repuser</code>
Trigger	<code>max_connections</code> raised on primary only (1500 → 3500)
Audience	Lead DBA / DBRE — execution-ready reference

Standby-first rule applies throughout · Keep this as a permanent reference

Contents

- 0 Scenario, Environment & Root Cause
- 1 Diagnostic Toolkit — WAL, LSN & Timeline
- 2 Decision Tree — Which Path Am I On?
- 3 Path A — WAL Present, Walreceiver Not Connecting
- 4 Path B — Rebuild Standby with `pg_basebackup`
- 5 Path C — Rebuild Standby with `pgBackRest`
- 6 Path D — `pg_rewind` after Timeline Divergence
- 7 Verification — Primary & Standby
- 8 Troubleshooting Playbook
- 9 Quick Command Reference Card

0 Scenario, Environment & Root Cause

Two days ago `max_connections` was raised on the **primary only** from 1500 to 3500. On the next primary restart, PostgreSQL emitted an `XLOG_PARAMETER_CHANGE` WAL record. The standby replayed that record, found its own `max_connections = 1500` insufficient for hot standby, and shut down with a FATAL. With the walreceiver gone, slot `ssncpri` on the primary flipped to `active = 'f'` and began **retaining WAL**. The standby is now behind by ~2 days.

The standby-first rule (why this happened): a small set of parameters must satisfy *standby value* $\geq$ *primary value* at all times. When you **increase** any of them you change the **standby first**, then the primary. When you **decrease** them you change the **primary first**, then the standby. This incident is the increase case done in the wrong order.

The five parameters governed by this rule:

Parameter	Rule on standby
<code>max_connections</code>	$\geq$ primary
<code>max_worker_processes</code>	$\geq$ primary
<code>max_wal_senders</code>	$\geq$ primary
<code>max_prepared_transactions</code>	$\geq$ primary
<code>max_locks_per_transaction</code>	$\geq$ primary

Environment used throughout this runbook (adjust hostnames/ports to your estate):

Item	Value
Primary	db01 (<primary_fqdn>), port 5432
Standby	db02 (<standby_fqdn>), port 5432
Replication user	repuser
Physical slot	ssncpri
Data directory	/apps/pgsql_data/16
WAL archive	/apps/pgsql_archives/
Logs	/apps/logs
Backups	/apps/backups
systemd unit	postgresql-16 (rename to match your unit)

Current state (per your update): the standby is now **up** with `max_connections = 3500` — the parameter block is cleared — but replication is still broken and the standby is far behind. That means we have moved past the parameter problem into a **WAL-gap / connection** problem, which is exactly what Section 1 diagnoses. Nothing is lost: while the primary is healthy it is the source of truth.

Run these before choosing a path. They tell you (a) whether the slot still holds the WAL the standby needs, (b) whether the standby is merely catching up, and (c) whether a timeline has diverged. Every command notes which node to run it on.

1.1 LSN basics — where is each node?

An **LSN** (Log Sequence Number, e.g. `3A/16B37C8`) is a byte position in the WAL stream. Replication health is just a comparison of LSNs between primary and standby.

```
-- ON PRIMARY: current write position
SELECT pg_current_wal_lsn() AS primary_lsn;
```

```
-- ON STANDBY: how far it has received vs replayed
SELECT pg_last_wal_receive_lsn() AS received_lsn, -- arrived over the wire
       pg_last_wal_replay_lsn()  AS replayed_lsn,  -- applied to data files
       pg_is_in_recovery()       AS in_recovery;   -- must be t on a standby
```

```
-- ON STANDBY: replication lag in human terms
SELECT now() - pg_last_xact_replay_timestamp() AS time_lag;
```

If `received_lsn` on the standby is advancing toward the primary's LSN, the standby is streaming and catching up — you are likely on **Path A (just wait)**. If it is frozen, keep diagnosing.

1.2 The replication slot — the single most important check

The slot decides everything. Its `wal_status` column is the fork between "recoverable by streaming" and "rebuild required".

```
-- ON PRIMARY
SELECT slot_name,
       active,           -- f = no standby attached
       active_pid,       -- NULL when inactive
       wal_status,       -- reserved | extended | unreserved | lost
       restart_lsn,      -- oldest WAL the slot still needs
       confirmed_flush_lsn,
       pg_size_pretty(
         pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
       ) AS retained_wal -- how much WAL the slot is pinning
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
```

wal_status	Meaning	Implication
reserved	Required WAL is within <code>max_wal_size</code> .	Streaming can resume → Path A.
extended	Exceeds <code>max_wal_size</code> but still retained by the slot / <code>wal_keep_size</code> .	Streaming can resume → Path A.
unreserved	Slot no longer guarantees the WAL; some segments will be removed at the next checkpoint.	Act now — reconnect immediately or rebuild.
lost	Required WAL has already been removed; the slot is unusable.	Streaming cannot resume → rebuild (Path B/C).

Why this fork exists: if `max_slot_wal_keep_size` is set, the slot is invalidated once retained WAL crosses that cap (you get `lost`, disk is safe, rebuild needed). If it is unset, the slot pins WAL forever — the slot stays healthy but your **primary disk** is the thing at risk.

1.3 Is the standby actually streaming? (both views)

```
-- ON PRIMARY: one row per connected standby. Empty = nobody streaming.
SELECT client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS byte_lag,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

```
-- ON STANDBY: the receiver side. Empty = walreceiver not running/connected.
SELECT status, -- streaming | starting | ...
       sender_host, sender_port,
       slot_name, -- must be ssncpri
       last_msg_receipt_time,
       latest_end_lsn
FROM pg_stat_wal_receiver;
```

`pg_stat_replication` empty on the primary and `pg_stat_wal_receiver` empty on the standby together mean: no stream. Combine with `wal_status` to decide Path A vs B/C.

1.4 Timeline ID — reading it on both nodes

The **timeline ID (TLI)** is a counter that increments every time a standby is **promoted**. A primary and its standby must share the same timeline to stream. A mismatch means divergence → only `pg_rewrite` or a full rebuild can reconcile them. The authoritative, offline way to read the TLI is `pg_controldata`.

```
# RUN ON EACH NODE (works while server is down too)
pg_controldata -D /apps/pgsql_data/16 | grep -Ei 'TimeLineID|REDO|checkpoint location|recovery ending'

# Key lines to compare between the two nodes:
# Latest checkpoint's TimeLineID:         <- the cluster's current timeline
# Latest checkpoint location:             <- LSN of last checkpoint
# Latest checkpoint's REDO location:      <- where recovery would start
# Min recovery ending location:           <- (standby) how far it must replay
# Min recovery ending loc's timeline:     <- (standby) timeline of that point
```

```
-- ONLINE equivalents (server up)
SELECT timeline_id FROM pg_control_checkpoint();           -- current TLI
SELECT pg_walfile_name(pg_current_wal_lsn());              -- ON PRIMARY: current WAL file
-- ON STANDBY:
SELECT pg_last_wal_replay_lsn(),
       pg_walfile_name(pg_last_wal_replay_lsn());
```

Decision rule: if Latest checkpoint's TimeLineID is the **same** on both nodes → no divergence (Path A/B/C). If the standby (or former standby) is on a **higher** timeline than the primary → it was promoted and diverged → **Path D** (pg_rewind).

1.5 Decoding a WAL filename (TLI + segment)

A WAL segment filename is 24 hex characters in three 8-char groups:

```
0000000A 00000016 000000B3
 \_____/ \_____/ \_____/
 timeline logical segment
  (TLI)   xlog id  number
```

```
# Example: 000000010000000000000003
# timeline    = 1
# logical id  = 0
# segment     = 3
# A *.history file such as 00000002.history documents the branch point
# where timeline 2 split off from timeline 1.
```

```
# Inspect WAL + history files on disk (either node)
ls -l /apps/pgsql_data/16/pg_wal/ | tail -20
ls -l /apps/pgsql_data/16/pg_wal/*.history 2>/dev/null
ls -l /apps/pgsql_archives/*.history 2>/dev/null

# Convert any LSN to its WAL filename on the current timeline
psql -c "SELECT pg_walfile_name('3A/16B37C8');"
```

1.6 Quantify the gap and the disk risk

```
-- ON PRIMARY: how far ahead of the slot is the live WAL? (= the backlog)
SELECT slot_name,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS gap
FROM pg_replication_slots WHERE slot_name='ssncpri';
```

```
# ON PRIMARY: is pg_wal filling the disk because the slot is pinning WAL?
df -h /apps/pgsql_data
du -sh /apps/pgsql_data/16/pg_wal
ls /apps/pgsql_data/16/pg_wal | wc -l # segment count (each is 16 MB by default)
```

Time-sensitive: an inactive slot retains WAL on the primary indefinitely (unless `max_slot_wal_keep_size` caps it). After ~2 days this can threaten the primary's disk. Confirm headroom **before** a long rebuild; if space is critical and you have decided to rebuild anyway, dropping the stale slot frees the pinned WAL immediately.

1.7 Confirm the original root cause in the logs

So you are certain you are not chasing a second, unrelated fault:

```
grep -Ei 'insufficient parameter|max_connections|hot standby is not possible|recovery has
paused|FATAL|started streaming|has already been removed|requested WAL segment' \
/apps/logs/*.log | tail -40
```

Signatures you may see, and what each maps to:

Log signature	Means	Go to
max_connections = 1500 is a lower setting...	Original parameter block (already fixed)	—
started streaming WAL from primary...	Standby reconnected, catching up	Path A
requested WAL segment ... has already been removed	Needed WAL is gone	Path B / C
new timeline N forked off... / highest timeline N...	Timeline divergence	Path D

2 Decision Tree — Which Path Am I On?

Read the three signals from Section 1 in this order. Stop at the first match.

#	Signal (from Section 1)	Conclusion	Go to
1	<code>pg_is_in_recovery()</code> returns f on the standby, or its TLI is higher than the primary's	Standby was promoted — timelines diverged	Path D
2	<code>wal_status</code> = reserved/extended and standby log shows started streaming WAL	WAL intact; standby is simply replaying the backlog	Path A (wait)
3	<code>wal_status</code> healthy but <code>pg_stat_wal_receiver</code> is empty (no stream starting)	WAL is fine; this is a connection / config fault	Path A (fix conn)
4	<code>wal_status</code> = lost , or log shows has already been removed	Required WAL gone; streaming can never resume from here	Path B / C

For this incident, signal #4 is the most likely outcome after ~2 days of an inactive slot, unless `max_slot_wal_keep_size` was unset and the primary disk held all the WAL (then #2/#3 apply). `pg_rewind` (Path D) only applies if the standby was ever promoted — in your scenario it was not, so Path D is included for completeness and future incidents.

3 Path A — WAL Present, Walreceiver Not Connecting

Use when `wal_status` is **reserved/extended** (the WAL the standby needs still exists) but the standby is not streaming. Two sub-cases.

3.A It IS streaming — just catching up

If `pg_stat_wal_receiver.status = streaming` and the standby log shows `started streaming WAL from primary`, there is nothing to fix. Watch the LSNs converge:

```
-- ON STANDBY (repeat every few seconds)
SELECT pg_last_wal_receive_lsn() AS recv,
       pg_last_wal_replay_lsn() AS replay,
       now() - pg_last_xact_replay_timestamp() AS lag;
```

```
-- ON PRIMARY: byte_lag should shrink toward 0
SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS byte_lag
FROM pg_stat_replication;
```

When `byte_lag` reaches near zero, the slot flips to `active = t` and the primary's `pg_wal` begins shrinking. Done.

3.B It is NOT connecting — fix the connection

If `pg_stat_wal_receiver` is empty but WAL is intact, it is an auth / network / config issue. Work the standby log top-down and check each link.

Step 1 — confirm the connection settings are intact (do **not** hand-edit these):

```
# ON STANDBY
grep -E 'primary_conninfo|primary_slot_name|recovery_min' \
  /apps/pgsql_data/16/postgresql.auto.conf
ls -l /apps/pgsql_data/16/standby.signal # must exist for a standby
```

`primary_slot_name` must read `'ssncpri'` and `primary_conninfo` must point at the primary with user `repuser`.

Step 2 — prove the standby can actually reach the primary as repuser:

```
# ON STANDBY: replication-protocol connectivity test
psql "host=<primary_fqdn> port=5432 user=repuser dbname=postgres \
      replication=database" -c "IDENTIFY_SYSTEM;"
# Returns systemid | timeline | xlogpos | dbname -> auth + network OK
# Errors here tell you exactly what to fix (auth, pg_hba, firewall, host).
```

Step 3 — on the PRIMARY, confirm the HBA rule and that the slot exists:

```
# ON PRIMARY
grep -E 'replication' /apps/pgsql_data/16/pg_hba.conf
# Expect a line allowing repuser from the standby IP, e.g.:
# host replication repuser <standby_ip>/32 scram-sha-256
```

```
-- ON PRIMARY
SELECT slot_name, active FROM pg_replication_slots WHERE slot_name='ssncpri';
```

Step 4 — after fixing whatever the log named, nudge the receiver:

```
-- ON STANDBY: restart the walreceiver without a full restart
SELECT pg_reload_conf();           -- if you changed primary_conninfo via ALTER SYSTEM
-- if needed, a clean restart re-establishes the stream:
-- sudo systemctl restart postgresql-16
```

Standby rule reminder: change `primary_conninfo` only via `ALTER SYSTEM SET primary_conninfo = '...'` then reload — never `ALTER SYSTEM RESET ALL` on the standby, which wipes it from `postgresql.auto.conf`.

It reconnects to the **existing** slot — no need to recreate `ssncpri`. Then run Section 7 to verify. If the log instead shows *WAL removed*, you are not on Path A — go to Path B/C.

4 Path B — Rebuild Standby with `pg_basebackup`

Use when `wal_status = lost` or the log shows `requested WAL segment ... has already been removed`. The standby must be re-seeded from the primary. **No data is lost** — the primary is the source of truth. This is the simplest, network-only rebuild.

4.1 Pre-flight

```
# ON PRIMARY: confirm it can serve a base backup
psql -c "SHOW max_wal_senders;"          -- must be > current senders (>= a few)
psql -c "SHOW wal_level;"                -- replica or logical (not minimal)
psql -c "SELECT pg_is_in_recovery();"      -- must be f (it is the primary)

# ON PRIMARY: pg_hba.conf must allow repuser replication from the standby IP
grep replication /apps/pgsql_data/16/pg_hba.conf
```

4.2 Stop the standby and preserve the old data dir

Rename rather than delete — an instant rollback if anything goes wrong.

```
# ON STANDBY
sudo systemctl stop postgresql-16

sudo mv /apps/pgsql_data/16 /apps/pgsql_data/16.broken.$(date +%Y%m%d_%H%M)
sudo -u postgres mkdir -p /apps/pgsql_data/16
sudo chmod 700 /apps/pgsql_data/16
```

4.3 Drop the stale/lost slot on the primary

So the rebuild can recreate it cleanly with `-C -S`.

```
-- ON PRIMARY
SELECT pg_drop_replication_slot('ssncpri'); -- frees the pinned WAL immediately
```

4.4 Re-seed and wire up replication in one shot

```
# ON STANDBY (run as the postgres user)
sudo -u postgres pg_basebackup \
  -h <primary_fqdn> -p 5432 -U repuser \
  -D /apps/pgsql_data/16 \
  -Fp \           # plain format (a normal data directory)
  -Xs \           # stream WAL during backup -> guaranteed consistent
  -P \           # progress reporting
  -R \           # write standby.signal + primary_conninfo + slot name
  -C -S ssncpri  # create the physical slot ssncpri on the primary
```

Flag	Why it matters here
-Xs	Streams the WAL generated <i>during</i> the copy on a second connection, so the backup is self-consistent without relying on the archive.
-R	Writes <code>standby.signal</code> and appends <code>primary_conninfo + primary_slot_name='ssncpri'</code> to <code>postgresql.auto.conf</code> .
-C -S ssncpri	Creates the slot on the primary as part of the run (that is why we dropped it in 4.3).
-Fp	Plain directory layout (vs <code>-Ft</code> tar). Use plain so you can start in place.

Bonus: the base backup copies the primary's `postgresql.conf`, so `max_connections` arrives as **3500** automatically — the original mismatch cannot reappear, and you do not hand-edit anything.

4.5 Start and watch

```
# ON STANDBY
sudo systemctl start postgresql-16
tail -f /apps/logs/*.log

# Healthy sequence in the log:
#   entering standby mode
#   consistent recovery state reached at ...
#   started streaming WAL from primary at .../... on timeline N
```

Then run Section 7 to verify. The first stream after a rebuild also re-arms the slot to `active = t` on the primary.

4.6 If the database is very large

A network `pg_basebackup` of a multi-hundred-GB cluster can take hours and re-copies everything. Two options: add `--checkpoint=fast` to start sooner, or use pgBackRest (Path C) which can do a **delta** restore and pull from `/apps/pgsql_archives/` rather than the live primary.

5 Path C — Rebuild Standby with pgBackRest

Same goal as Path B, but the seed comes from your pgBackRest repository instead of a live copy off the primary. Faster for large clusters (supports `--delta`), and it offloads the primary by reading the repo `/apps/pgsql_archives/`. Replace `<stanza>` with your stanza name.

Key difference from Path B: a pgBackRest restore does **not** create the replication slot. You must create `ssncpri` on the primary yourself (Step 5.3).

5.1 Pre-flight — verify the repo has a usable backup

```
# ON STANDBY (or wherever pgBackRest is configured)
pgbackrest --stanza=<stanza> info
# Confirm a recent full/incr backup exists and the WAL archive is continuous.
# Check the stanza + repo path in /etc/pgbackrest/pgbackrest.conf
```

5.2 Stop the standby and preserve the old data dir

```
# ON STANDBY
sudo systemctl stop postgresql-16
sudo mv /apps/pgsql_data/16 /apps/pgsql_data/16.broken.$(date +%Y%m%d_%H%M)
sudo -u postgres mkdir -p /apps/pgsql_data/16
sudo chmod 700 /apps/pgsql_data/16
```

5.3 Recreate the slot on the primary (pgBackRest will not)

```
-- ON PRIMARY: drop the lost slot, then create a fresh physical slot
SELECT pg_drop_replication_slot('ssncpri')
WHERE EXISTS (SELECT 1 FROM pg_replication_slots WHERE slot_name='ssncpri');
SELECT pg_create_physical_replication_slot('ssncpri');
```

5.4 Restore as a standby

`--type=standby` writes `standby.signal` and recovery settings. `--delta` only copies blocks that differ — safe and fast when reusing an existing data dir (omit it here since we emptied the directory; keep it if you skip the rename and restore in place).

```
# ON STANDBY (as postgres)
sudo -u postgres pgbackrest \
  --stanza=<stanza> \
  --type=standby \
  --recovery-option=primary_conninfo="host=<primary_fqdn> port=5432 user=repuser" \
  --recovery-option=primary_slot_name=ssncpri \
  restore

# Add --delta if restoring in place over an existing directory:
# ... --delta --type=standby ... restore
```

Option	Effect
<code>--type=standby</code>	Creates <code>standby.signal</code> so the node comes up in continuous recovery.
<code>--recovery-option=primary_conninfo=...</code>	Written into <code>postgresql.auto.conf</code> so it streams from the primary after replaying the archive.
<code>--recovery-option=primary_slot_name=ssncpri</code>	Binds the new standby to the slot you created in 5.3.
<code>--delta</code>	Block-level diff restore; skips unchanged files. Use only when restoring over an existing dir.

5.5 Confirm recovery wiring, then start

```
# ON STANDBY: verify pgBackRest wrote the expected settings
grep -E 'primary_conninfo|primary_slot_name|restore_command' \
  /apps/pgsql_data/16/postgresql.auto.conf
ls -l /apps/pgsql_data/16/standby.signal

sudo systemctl start postgresql-16
tail -f /apps/logs/*.log
# Expect: restoring from archive -> consistent recovery -> started streaming WAL
```

Two-stage recovery is normal here: the standby first replays WAL from the archive (`restore_command`), catches up to near-current, then switches to live streaming via the slot. You will see archive restores in the log *before* the `started streaming WAL` line.

Then run Section 7 to verify. Also re-confirm `max_connections = 3500` on the restored standby (pgBackRest restores the primary's config, so it should match):

```
psql -c "SHOW max_connections;" -- expect 3500
```

6 Path D — `pg_rewind` after Timeline Divergence

Use **only** when the standby was **promoted** (TLI higher than the primary, or `pg_is_in_recovery() = f`) and you want to re-attach it as a standby **without** a full re-seed. `pg_rewind` rewinds the diverged node back to the last common checkpoint, then it catches up by streaming. This did **not** happen in your incident — included for future failovers / DR drills.

Prerequisite — check before you rely on this: the node you rewind must have been initialised with **data checksums** or have `wal_log_hints = on`. Without one of those, `pg_rewind` cannot work and you fall back to Path B/C.

6.1 Confirm divergence and rewind viability

```
# On BOTH nodes: compare timelines
pg_controldata -D /apps/pgsql_data/16 | grep -i "TimelineID"
#   target (to be rewound) on a HIGHER timeline than the new primary => diverged

# Check rewind prerequisites on the TARGET (the node you will rewind)
pg_controldata -D /apps/pgsql_data/16 | grep -i "checksum" # "Data page checksum version: 1"
= on
psql -c "SHOW wal_log_hints;" # on = good
```

6.2 Identify the roles

After a failover the picture is usually: **db02** was promoted (new primary, higher TLI); **db01** (old primary) is to be rewound and re-attached as the new standby. Map the commands to whichever node is the rewind *target*.

6.3 Cleanly shut down the rewind target

`pg_rewind` requires the target to have been stopped **cleanly** (not crash-stopped).

```
# ON TARGET (the node to be rewound)
sudo systemctl stop postgresql-16
# verify clean shutdown:
pg_controldata -D /apps/pgsql_data/16 | grep -i "Database cluster state"
#   want: "shut down" (if "in production" -> start then stop once more to get clean)
```

6.4 Run `pg_rewind`

Prefer `--source-server` (live source over libpq) so it can fetch needed blocks directly. The source-side user needs rights to read WAL/control data and run the needed functions.

```
# ON TARGET, as postgres
sudo -u postgres pg_rewind \
  --target-pgdata=/apps/pgsql_data/16 \
  --source-server="host=<new_primary_fqdn> port=5432 user=repuser dbname=postgres" \
  --progress \
  --write-recovery-conf # writes standby.signal + primary_conninfo (PG 13+)

# Dry run first to see what it WOULD do, changing nothing:
#   sudo -u postgres pg_rewind --target-pgdata=/apps/pgsql_data/16 \
#     --source-server="..." --dry-run --progress
```

Option	Effect
<code>--target-pgdata</code>	The diverged data dir to rewind (the new standby).
<code>--source-server</code>	libpq conn to the new primary; lets <code>pg_rewind</code> pull blocks it needs.
<code>--write-recovery-conf</code>	Writes <code>standby.signal</code> + <code>primary_conninfo</code> so it boots as a standby.
<code>--dry-run</code>	Reports the plan without modifying anything — always do this first.

6.5 Set the slot, start, verify

```
-- ON NEW PRIMARY: ensure the slot exists for the rewound standby
SELECT pg_create_physical_replication_slot('ssncpri')
WHERE NOT EXISTS (SELECT 1 FROM pg_replication_slots WHERE slot_name='ssncpri');
```

```
# ON TARGET: add slot name (pg_rewind does not set primary_slot_name), then start
sudo -u postgres bash -c "echo \"primary_slot_name = 'ssncpri'\" >> \
/apps/pgsql_data/16/postgresql.auto.conf"
sudo systemctl start postgresql-16
tail -f /apps/logs/*.log      # expect: started streaming WAL ... on timeline N
```

If `pg_rewind` reports "could not find common ancestor" or the checksum/`wal_log_hints` prerequisite is unmet, abandon Path D and rebuild via Path B/C — the outcome is identical, just slower.

7 Verification — Primary & Standby

Run after any path. Replication is healthy when: primary shows the standby as `streaming`, the slot is `active = t`, lag trends to ~0, and the standby confirms it is in recovery and receiving on slot `ssncpri`.

7.1 On the PRIMARY

```
-- (a) slot is live again
SELECT slot_name, active, active_pid, wal_status, restart_lsn
FROM pg_replication_slots WHERE slot_name='ssncpri';
-- expect: active = t, active_pid set, wal_status = reserved/extended

-- (b) standby is streaming, lag shrinking
SELECT client_addr, application_name, state, sync_state,
       sent_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS byte_lag,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
-- expect: state = streaming, byte_lag falling toward 0

-- (c) pg_wal is no longer growing (slot is advancing -> disk risk closed)
SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS still_pinned
FROM pg_replication_slots WHERE slot_name='ssncpri';
```

7.2 On the STANDBY

```
-- (a) it is a standby and replaying
SELECT pg_is_in_recovery(); -- expect t

-- (b) receiver is connected on the right slot
SELECT status, sender_host, slot_name, latest_end_lsn, last_msg_receipt_time
FROM pg_stat_wal_receiver; -- status = streaming, slot_name = ssncpri

-- (c) receive vs replay vs time lag
SELECT pg_last_wal_receive_lsn() AS recv,
       pg_last_wal_replay_lsn() AS replay,
       now() - pg_last_xact_replay_timestamp() AS time_lag;

-- (d) parameter is correct (the original trigger)
SHOW max_connections; -- expect 3500
```

7.3 End-to-end write test (optional, definitive)

Proves a real change flows primary → standby.

```
-- ON PRIMARY
CREATE TABLE IF NOT EXISTS repl_check(id int, ts timestampz default now());
INSERT INTO repl_check(id) VALUES (1);
SELECT pg_current_wal_lsn();
```

```
-- ON STANDBY (a moment later) -- read-only, should return the row
SELECT * FROM repl_check ORDER BY ts DESC LIMIT 1;
-- A write attempt here must fail with "cannot execute INSERT in a read-only transaction"
-- which correctly confirms it is a standby.
```

```
-- ON PRIMARY: clean up  
DROP TABLE repl_check;
```

7.4 Timeline sanity (both nodes share one timeline)

```
# Should match on both nodes once re-synced  
psql -c "SELECT timeline_id FROM pg_control_checkpoint();"  
# or offline:  
pg_controldata -D /apps/pgsql_data/16 | grep -i "TimeLineID"
```

8 Troubleshooting Playbook

Symptom / log line	Cause	Fix
<code>max_connections = 1500</code> is a lower setting than on the primary	Parameter mismatch (the original break)	Set standby $\geq$ primary, restart standby (already done: 3500).
<code>requested WAL segment ...</code> has already been removed	Slot lost; needed WAL purged	Rebuild via Path B or C.
<code>wal_status = lost</code>	<code>max_slot_wal_keep_size</code> cap crossed	Rebuild (B/C); reconsider the cap value.
replication slot "ssncpri" already exists	Slot present when <code>pg_basebackup -C</code> tried to create it	Drop it first: <code>pg_drop_replication_slot('ssncpri')</code> .
FATAL: password authentication failed for user "repuser"	Bad creds / wrong auth method	Check <code>.pgpass</code> , <code>pg_hba.conf</code> method (scram-sha-256), repuser password.
no <code>pg_hba.conf</code> entry for replication connection	Missing HBA replication rule	Add <code>host replication repuser <standby_ip>/32 scram-sha-256</code> , reload.
could not connect to server: Connection refused / timeout	Firewall / port / wrong host	Open 5432 standby→primary; verify <code>listen_addresses</code> and FQDN.
<code>pg_stat_wal_receiver</code> empty, slot reserved	Walreceiver not started/connecting	Path A.3.B: check <code>conninfo</code> , <code>IDENTIFY_SYSTEM</code> , restart standby.
target server must be shut down cleanly (<code>pg_rewind</code>)	Crash-stopped target	Start, then stop once cleanly; retry <code>pg_rewind</code> .
<code>pg_rewind</code> : cannot be enabled (no checksums / <code>wal_log_hints</code> off)	Rewind prerequisite unmet	Use Path B/C instead.
<code>pg_wal</code> filling the primary disk	Inactive slot pinning WAL	If rebuilding, drop the stale slot to free WAL immediately; then re-seed.
consistent recovery state reached, but no streaming	Archive replay ok, stream not started	Check <code>primary_slot_name</code> , slot exists on primary, conn reachable.

Golden rules carried through every path: (1) Standby-first when increasing the five governed parameters, primary-first when decreasing. (2) Never `ALTER SYSTEM RESET ALL` on the standby — it erases `primary_conninfo` from `postgresql.auto.conf`; use individual `ALTER SYSTEM RESET parameter_name`. (3) Rename, never delete, the old data dir during a rebuild. (4) The primary is the source of truth — a behind/empty standby is a sync problem, not data loss.

9 Quick Command Reference Card

One-glance cheatsheet. [P] = run on primary, [S] = run on standby.

Diagnose

```
[P] SELECT slot_name,active,wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),restart_lsn)) gap
FROM pg_replication_slots WHERE slot_name='ssncpri';
[P] SELECT * FROM pg_stat_replication;
[S] SELECT * FROM pg_stat_wal_receiver;
[S] SELECT pg_is_in_recovery(), now()-pg_last_xact_replay_timestamp() AS lag;
[*] pg_controldata -D /apps/pgsql_data/16 | grep -i TimeLineID
[*] grep -Ei 'streaming|removed|FATAL|insufficient' /apps/logs/*.log | tail
```

Path A — reconnect

```
[S] grep -E 'primary_conninfo|primary_slot_name' \
     /apps/pgsql_data/16/postgresql.auto.conf
[S] psql "host=<primary> user=repuser dbname=postgres replication=database" \
     -c "IDENTIFY_SYSTEM;"
[S] sudo systemctl restart postgresql-16
```

Path B — pg_basebackup rebuild

```
[S] sudo systemctl stop postgresql-16
[S] sudo mv /apps/pgsql_data/16 /apps/pgsql_data/16.broken.$(date +%F_%H%M)
[S] sudo -u postgres mkdir -p /apps/pgsql_data/16 && sudo chmod 700 /apps/pgsql_data/16
[P] SELECT pg_drop_replication_slot('ssncpri');
[S] sudo -u postgres pg_basebackup -h <primary> -p 5432 -U repuser \
     -D /apps/pgsql_data/16 -Fp -Xs -P -R -C -S ssncpri
[S] sudo systemctl start postgresql-16
```

Path C — pgBackRest rebuild

```
[*] pgbackrest --stanza=<stanza> info
[S] sudo systemctl stop postgresql-16 && \
     sudo mv /apps/pgsql_data/16 /apps/pgsql_data/16.broken.$(date +%F_%H%M) && \
     sudo -u postgres mkdir -p /apps/pgsql_data/16 && sudo chmod 700 /apps/pgsql_data/16
[P] SELECT pg_create_physical_replication_slot('ssncpri');
[S] sudo -u postgres pgbackrest --stanza=<stanza> --type=standby \
     --recovery-option=primary_conninfo="host=<primary> port=5432 user=repuser" \
     --recovery-option=primary_slot_name=ssncpri restore
[S] sudo systemctl start postgresql-16
```

Path D — pg_rewind (post-promotion only)

```
[T] sudo systemctl stop postgresql-16 # clean shutdown
[T] sudo -u postgres pg_rewind --target-pgdata=/apps/pgsql_data/16 \
     --source-server="host=<new_primary> port=5432 user=repuser dbname=postgres" \
     --progress --write-recovery-conf --dry-run # then re-run without --dry-run
[P] SELECT pg_create_physical_replication_slot('ssncpri');
[T] echo "primary_slot_name='ssncpri'" >> /apps/pgsql_data/16/postgresql.auto.conf
[T] sudo systemctl start postgresql-16
```

Verify

```
[P] SELECT slot_name,active,wal_status FROM pg_replication_slots WHERE slot_name='ssncpri';
[P] SELECT client_addr,state,
      pg_size_pretty(pg_wal_lsn_diff(sent_lsn,replay_lsn)) byte_lag
      FROM pg_stat_replication;
[S] SELECT pg_is_in_recovery();
[S] SELECT status,slot_name FROM pg_stat_wal_receiver;
[S] SHOW max_connections;    -- 3500
```

Runbook end. Built for PostgreSQL 16 physical streaming replication · slot `ssncpri` · user `repuser`. Substitute `<primary_fqdn>`, `<new_primary>`, `<standby_ip>` and `<stanza>` with your real values before execution.